PS-06: Smart Recycling Assistant — PRD & TRD

Hackathon track: Computer Vision + Digital Twin + Generative AI

Part 1: Product Requirements Document (PRD)

1. Overview

Product name: CircuLens

One-line pitch: Point a camera at any item and instantly learn whether it is recyclable, reusable, biodegradable or hazardous, while a digital twin tracks where materials flow and GenAI coaches you toward a circular economy.

Problem: People and institutions misclassify waste constantly. Contaminated recycling streams get rejected, hazardous items end up in general waste, and reusable goods are thrown away. Existing guidance is generic, local rules vary, and nobody sees the material flow end to end.

2. Goals and Non-Goals

Goals

- Recognize common waste items from an image/camera feed and classify into four categories: recyclable, reusable, biodegradable, hazardous.
- Identify the material and the correct bin or disposal route.
- Maintain a digital twin of material flow (item → bin → collection → processing → outcome) with live counts.
- Use GenAI to give personalized, local, context-aware recommendations and to explain "why".
- Show impact metrics: diversion rate, contamination rate, estimated CO2 saved.

Non-goals (for the hackathon)

- Real municipal logistics integration.

- Recognizing every possible object; focus on 15–30 common classes.
- Physical sorting hardware (optional stretch).

3. Target Users

Persona	Need
Student / resident	Quick answer on how to dispose of an item
Campus / society sustainability manager	See what is being thrown where, and contamination hotspots
Waste operator	Understand stream composition and plan collection

4. User Stories

1. As a user, I want to scan an item and immediately know which bin it goes in and why.
2. As a user, I want tips on reusing or repairing an item instead of discarding it.
3. As a manager, I want a live twin of bins and material flow to see fill levels and contamination.
4. As a manager, I want a report showing recycling rate over time and recommendations to improve it.
5. As a user, I want a personal dashboard of my recycling impact and suggestions for improvement.
6. As a manager, I want to simulate the effect of adding another bin or changing signage on diversion rate.

5. Functional Requirements

ID	Requirement	Priority
F1	Image/webcam upload and real-time classification of items	Must
F2	Four-way category output plus material type and confidence	Must

ID	Requirement	Priority
F3	Bin recommendation based on category and local rules	Must
F4	Digital twin of material flow: bins, counts, fill levels, contamination	Must
F5	Logging of every scan as a flow event	Must
F6	GenAI explanations and personalized tips (reuse, repair, donate, safe disposal for hazardous)	Must
F7	Dashboard: diversion rate, contamination, category breakdown, CO2 saved	Must
F8	What-if simulation on the twin	Should
F9	Low-confidence handling: ask user or fall back to GenAI vision	Should
F10	Chat assistant for recycling questions	Could
F11	Gamification: streaks, points, leaderboard	Could
F12	Multi-item detection in a single image	Could

6. Non-Functional Requirements

- **Latency:** classification result in under 1.5 s on a laptop/phone browser.
- **Accuracy:** top-1 accuracy ≥ 85% on the supported classes; hazardous items prioritize recall.
- **Safety:** for hazardous or uncertain items, always show a cautious message rather than a confident wrong one.
- **Usability:** works on mobile browser; result readable in one glance.

7. Success Metrics

- Classification accuracy ≥ 85% (hazardous recall ≥ 95%).
- Mean time from scan to answer < 2 s.
- Twin updates within 1 s of scan.

- Demonstrable improvement in simulated diversion rate after following recommendations.
- Judges understand the full loop (scan → twin → GenAI advice) in a 3-minute demo.

8. Demo Scenario

Scan a plastic bottle (recyclable), a banana peel (biodegradable), a used battery (hazardous) and a working old phone (reusable). The twin lights up each flow path, the dashboard updates, GenAI explains each decision and suggests donating the phone, and the what-if panel shows the diversion rate rising with an added e-waste bin.

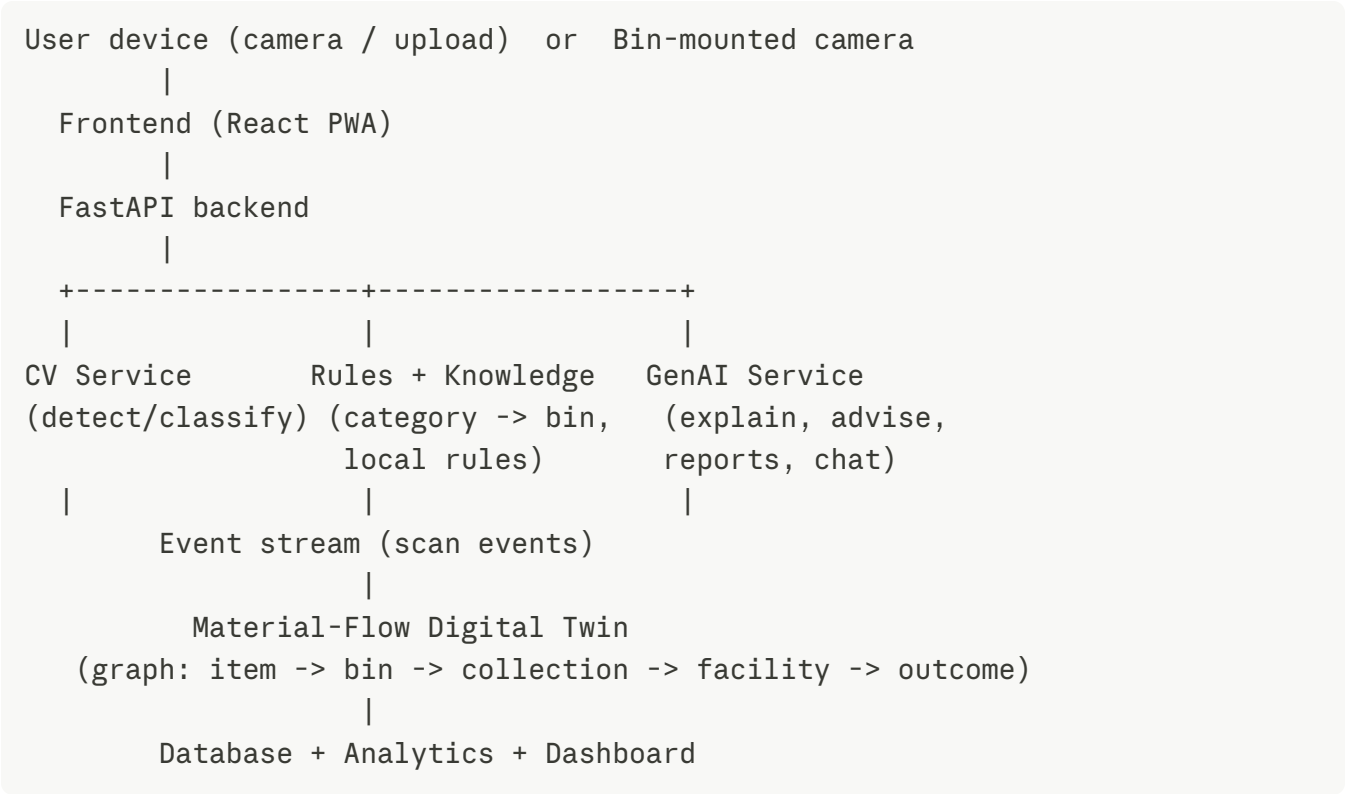
9. Risks

Risk	Mitigation
Limited labelled data	Combine public datasets, then fine-tune on your own photos
Visually similar classes (e.g. paper cup vs plastic cup)	Merge or refine classes; use GenAI vision as a second opinion on low confidence
Local rules differ	Keep rules in an editable config; state the assumed locality
Wrong hazardous advice	Rule-based override for hazardous items, plus caution text

---

Part 2: Technical Requirements Document (TRD)

1. System Architecture



2. Tech Stack

Layer	Choice	Notes
CV model	YOLOv8/YOLO11 (detect) or EfficientNet/MobileNetV3 (classify)	Classification is simpler; detection handles multiple items
Datasets	TrashNet, TACO, Waste Classification (Kaggle), plus own photos	Map labels to the four categories
Training	PyTorch + Ultralytics, Colab/Kaggle GPU	Fine-tune pretrained weights
Backend	FastAPI	REST + WebSocket

Layer	Choice	Notes
Twin	Graph model with NetworkX + simulation in Python; visualize via React Flow or Sankey (Plotly/D3)	Sankey is ideal for material flow
DB	SQLite → PostgreSQL	
GenAI	Claude API with image input as fallback and for explanations	Structured JSON output
Frontend	React PWA, Tailwind, Recharts	Camera access via getUserMedia
Deployment	Docker Compose or a single cloud VM	Keep an offline demo mode

3. Component Specifications

3.1 Computer Vision Service

- Input: JPEG frame (≤ 1280 px), preprocessed and resized.
- Model output: `{label, material, category, confidence, bbox?}`.
- Taxonomy: 15–30 item labels mapped to material (plastic, glass, paper, metal, organic, e-waste, battery, textile, etc.) and to the four categories.
- Confidence policy: ≥ 0.75 accept; 0.4–0.75 show top-3 and ask for confirmation; < 0.4 send to GenAI vision fallback.
- Optimization: ONNX export or TorchScript; quantize for CPU inference.

3.2 Rules and Knowledge Base

- JSON/YAML config: `category → bin_color, bin_name, preparation_steps, notes`.
- Hazard override list: batteries, bulbs, aerosols, chemicals, medical waste always route to hazardous handling with a safety note.
- Locality profile (for example, a generic Indian city profile with wet/dry/hazardous bins), editable.

3.3 Digital Twin (Material Flow)

- Nodes: items/categories, bins (with capacity), collection routes, processing facilities (recycler, composter, e-waste handler, landfill).

- Edges: flow with counts/weight and contamination flag.
- State update: each scan event increments the flow from category to the chosen bin; contamination recorded when a user bin choice conflicts with the recommended one.
- Estimated weight per item class from a lookup table (for example, PET bottle ≈ 20 g).
- Fill-level model: `fill = sum(item_volume) / bin_capacity`.
- Simulation: scenarios (add a bin, change signage, increase compost pickup) shift probabilities of correct disposal; output projected diversion rate, contamination rate and CO2e avoided.
- CO2 factors: per-material avoided-emission factors table (document the source and treat the figures as estimates).

3.4 GenAI Service

- **Per-scan explanation:** short "why this bin" and one reuse/repair tip.
- **Personalization:** based on the user's scan history (for example, frequent soft-plastic mistakes) produce 3 targeted tips.
- **Manager report:** summarizes twin metrics, hotspots and top recommended interventions.
- **Vision fallback:** send the image with a strict JSON schema asking for item, material, category and certainty.
- **Guardrails:** category and bin come from the rules engine, not from free-form generation. GenAI explains and advises but does not override hazard rules. Use computed numbers only.

3.5 Backend API

Endpoint	Method	Purpose
`/classify`	POST	Image in, classification and bin out
`/scan/confirm`	POST	Confirm or correct a result and log the flow event
`/twin/state`	GET	Current graph, fill levels, flows
`/twin/simulate`	POST	What-if scenario
`/insights/user/{id}`	GET	Personalized tips
`/report`	POST	Manager report

Endpoint	Method	Purpose
`/chat`	POST	Recycling Q&A
`/ws/live`	WS	Live twin updates

3.6 Frontend

- Scan screen: camera view, bounding box, result card (category colour, bin, confidence, tip).
- Twin view: Sankey/graph of flows and animated bin fill levels.
- Insights: personal impact, contamination breakdown.
- Manager tab: simulation controls and generated report.

4. Data Model (core tables)

- `items(id, label, material, category, avg_weight_g, co2_factor)`
- `bins(id, name, category, capacity_l, location)`
- `scan_events(id, user_id, ts, item_id, predicted_category, confidence, chosen_bin_id, correct, image_ref?)`
- `flow_edges(id, source, target, count, weight_kg, contaminated_count)`
- `users(id, name, points, streak)`
- `reports(id, generated_at, payload_json)`

5. Model Development Plan

1. Define taxonomy and the label → category mapping.
2. Merge TrashNet/TACO/Kaggle data; clean and balance classes.
3. Fine-tune a pretrained model (MobileNetV3 or YOLO-nano); use augmentation (lighting, rotation, background).
4. Add 100+ of your own photos per weak class (phone photos, real bins).
5. Evaluate with a confusion matrix; prioritize hazardous recall.
6. Export to ONNX and benchmark latency.

6. Performance and Deployment

- Target: < 300 ms inference on CPU for a classifier; < 1.5 s end-to-end including the network.
- Cache GenAI explanations per label so repeated scans are instant.
- Pre-seed the twin with synthetic historical data so the dashboard is rich on demo day.

7. Security and Privacy

- Images processed transiently; store only the label and metadata unless the user opts in.
- API keys server-side only.
- Avoid capturing faces: show a hint to frame only the item; optionally blur non-item regions.

8. Testing

- Model: per-class precision/recall, hazardous recall target, test on photos from different environments.
- Rules: unit tests for every category-to-bin mapping and hazard override.
- Twin: replay a scripted sequence of scans and assert flow counts and fill levels.
- GenAI: schema validation; verify no category contradicts the rules engine.

9. Suggested Timeline (48-hour example)

Block	Work
Hours 0–6	Taxonomy, dataset prep, backend skeleton, rules config
Hours 6–18	Train and evaluate the model, build `/classify`
Hours 18–28	Frontend scan flow, scan logging, twin graph and Sankey
Hours 28–38	GenAI explanations, personalization, report, simulation

Block	Work
Hours 38–44	Fallback handling, polish, seed data, mobile testing
Hours 44–48	Demo rehearsal and pitch

10. Stretch Goals

- Multi-item detection with per-item bin routing.
- Raspberry Pi smart-bin prototype with a camera and servo lid.
- Barcode/packaging-symbol reading (resin codes) for finer material identification.
- Campus-wide leaderboard and challenges.